Composer Classification from Symbolic Music

UE24CS352A Machine Learning · Mini-project · Section C, Team 10

Vivian Sobers E PES1UG24CS901

C Nitish PES1UG25CS809

github.com/VivianSobers/composer-classification

1. Problem statement

Problem statement #67: given only the MIDI encoding of a piece of Western classical music, predict its composer, and find which musical properties separate composers best. We reimplement and re-evaluate Jain, Smit and Yngesjö, *Analysis and Classification of Symbolic Western Classical Music by Composer* (Stanford CS229, 2019). Their dataset of about 10,000 MIDI files was never released, so we use public data and our numbers are not directly comparable with theirs.

2. Dataset

- **midi-classical-music** (Hugging Face): 4,796 MIDI files of piano, chamber and orchestral works. After removing duplicate files and keeping composers with at least 50 files and 5 distinct pieces, 2,997 files from 14 composers remain. Classes are unbalanced: one test split holds 203 Bach files but only 8 by Clementi.
- **MAESTRO v3.0.0**: 1,276 piano performances; 6 composers pass the same filter. Its official split keeps each composition in one subset. We use it for the piano-roll CNN and to compare it with the feature-based models on the same data.

3. Approach

Each file gets a piece ID shared by all movements and copies of one work. Train, dev and test splits are grouped by this ID, so the same work never appears on both sides. jSymbolic 2.2 extracts 1,495 features per piece covering pitch, melodic and vertical intervals, rhythm, texture, instrumentation and dynamics; music21's extractors are the fallback. Six classifiers are compared: LightGBM, softmax regression, an RBF SVM, LDA, k-NN and an MLP. Each gets the same budget of eight hyperparameter settings; the one with the best dev macro-F1 is kept, and the test set is scored once. Every run is repeated for seeds 0 to 4 and we report the mean and standard deviation. We also train the paper's CNN on 64 × 1536 piano-roll images of MAESTRO, with and without augmentation. The experiments are:

- **E1** six classifiers on the 14 composers with grouped splits; **E2** the same with a random file-level split, as in the paper, to measure leakage;
- **E3** LightGBM on one feature group at a time, for composer and for era (Baroque, Classical, Romantic);
- **E4** the six classifiers on MAESTRO's official split; **E5** the piano-roll CNN on the same split.

Differences from the paper: we group splits by piece, fit the scaler on training data only, give every feature model the same tuning budget, and score the CNN in `eval()` mode with the epoch chosen on dev. The CNN was not tuned and ran on CPU because the GPU servers were busy.

4. Implementation

The code is one Python 3.13 package, `composer_clf`, with one command per step: `data.download` and `data.catalog` fetch the datasets, remove duplicates and assign piece IDs; `features.extract` runs jSymbolic in parallel Java processes and retries files that run out of heap; `run_tabular` runs E1 to E4 and `cnn.run` runs E5. Each run saves its config (with git commit and library versions), metrics, dev tuning table and test predictions, and `report` rebuilds every table and chart from those files. The demo is a Streamlit app: upload a MIDI file and it shows the predicted composer, the probability of each composer, the features that pushed the prediction most and the piano roll. The repository has 98 pytest tests. Libraries: scikit-learn, LightGBM, PyTorch, music21, pretty_midi, pandas and matplotlib.

5. Results

Model	E1 accuracy	E1 macro-F1	E2 accuracy (random)	E4 MAESTRO acc.
LightGBM	**0.787 ± 0.012**	**0.615 ± 0.033**	**0.867 ± 0.010**	**0.772 ± 0.021**
MLP	0.736 ± 0.018	0.562 ± 0.028	–	0.686 ± 0.027
SVM (RBF)	0.728 ± 0.023	0.543 ± 0.035	0.830 ± 0.014	0.653
Softmax	0.726 ± 0.020	0.553 ± 0.034	0.831 ± 0.018	0.669
LDA	0.705 ± 0.017	0.524 ± 0.017	–	0.727
k-NN	0.627 ± 0.015	0.414 ± 0.021	–	0.603
CNN / with augmentation	–	–	–	0.545 / 0.559

Table 1. Test results, mean ± std over five seeds. On MAESTRO's fixed split, LDA, softmax, SVM and k-NN train deterministically, so their spread is zero.

LightGBM is the best model in every setting. Macro-F1 sits well below accuracy because the small classes are hard: Bach, Maier, Scarlatti and Vivaldi are recognised 95% of the time or more, while none of the 8 Clementi or 11 Schumann test pieces is classified correctly, and two thirds of Handel and Mendelssohn pieces are labelled Bach. A random split, as used by the paper, adds 8 to 10 points to every model (Figure 1), because movements and copies of one work land in both train and test.

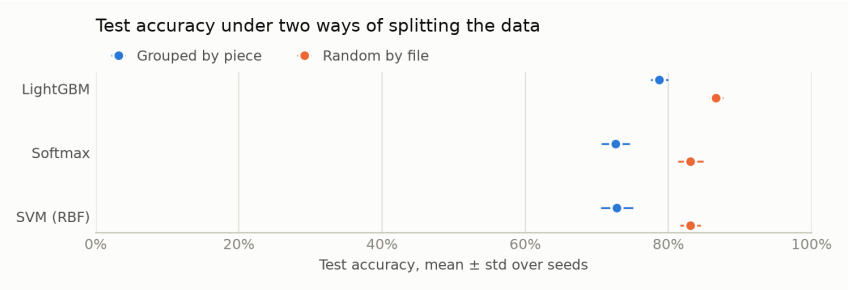


Figure 1. Test accuracy with grouped and random splits (E1 vs E2).

With one feature group at a time, LightGBM reaches 0.702 with rhythm, 0.675 with vertical intervals, 0.666 with texture, 0.639 with melodic intervals and 0.574 with instrumentation. Leaving out instrumentation gives 0.780, so the model does not rely on which instruments a file uses. Era classification is much easier: 0.960 with all features and 0.848 to 0.879 with any single group. Microtone Prevalence and Glissando Prevalence are among the most useful features, although classical scores contain almost none; we suspect they pick up pitch-bend messages left by whoever sequenced a file. On MAESTRO the CNN reaches only 0.545 (0.559 with augmentation) against 0.772 for LightGBM. Its training accuracy is 0.91 (0.99 with augmentation), so it memorises the 650 training performances.

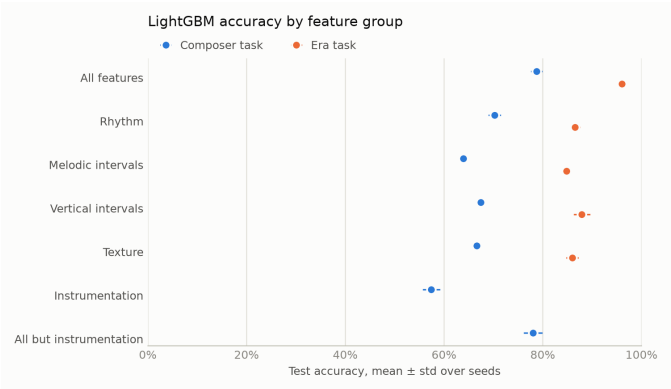


Figure 2. LightGBM accuracy with one feature group at a time, for composer and for era (E3).

6. Conclusions

- Composer can be predicted from MIDI alone: LightGBM on jSymbolic features reaches 0.787 accuracy on 14 composers and 0.772 on MAESTRO's 6.
- Splits must be grouped by piece. A random file split overstates accuracy by 8 to 10 points, so results reported that way, including the original paper's, are optimistic.
- Rhythm features carry the most composer information, and era is far easier to predict than composer. Some strong features may describe how a file was made rather than the music.
- At this data size, hand-built features beat the paper's CNN, which overfits. The CNN was untuned, so this is a lower bound for it; tuning it on a GPU and balancing the classes are the clearest next steps.